Studyroom: An AI Study Tutor That Only Answers From Your Files

The problem

When you study for an exam, your material is scattered: lecture PDFs, slide decks, recorded videos. Ask a general AI tutor a question and it happily mixes in whatever it knows from the internet. Sometimes it makes things up. And you can't tell which sentence came from your course and which didn't.

For studying, that is the worst combination. You can't trust the answer, and you can't check it either.

What we built

Studyroom is a study tutor that only knows what you give it. You upload your lecture PDFs, slide decks and videos, tick the ones you want to study from, and everything else (the tutor, quizzes, flashcards, topic map and progress view) works only on those files.

Every answer says where it came from, for example Lecture 1, Page 12 or Video 03:25. If your files don't contain the answer, the tutor says so instead of guessing.

What makes it different:

- **Your files only.** Files you didn't tick never leak into an answer. We test this by asking the same question with and without the right file selected.
- **Every claim has a source.** Answers, quiz questions and flashcards all carry a page, slide or timestamp.
- **It adapts.** Quiz results update a mastery score for each topic, and that score sets the next quiz's difficulty and points you at weak topics.
- **No sessions.** The selection is just a list of file ids sent with each request, so the backend stays simple.

What you can do with it

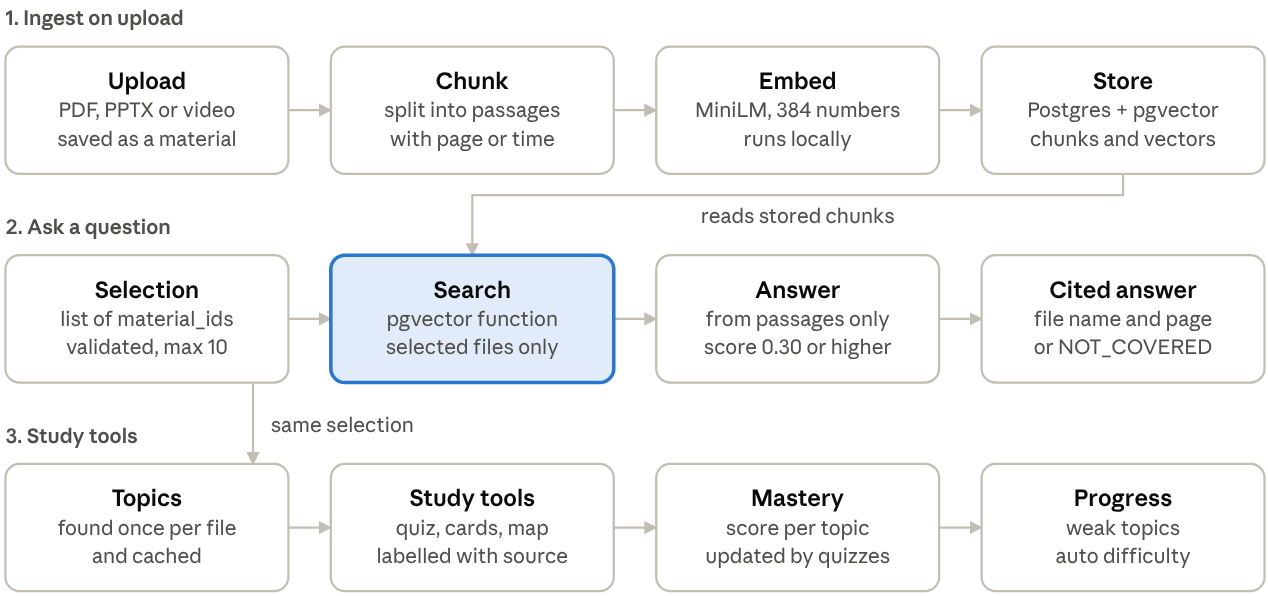
Studyroom takes PDFs (cited by page), PowerPoint decks (cited by slide) and videos (cited by timestamp). You can select up to 10 files at once.

Feature	What it does
Ask your tutor	Answers using only passages from the selected files. Replies "not covered" when they don't contain the answer.
Practice quiz	Multiple-choice questions by topic. Difficulty is easy, medium, hard or auto, which follows your mastery.
Flashcards	Mixed, single-topic or weak-topics-only decks. You mark each card known or not known.
Topic map	A mind map of topics and sub-ideas, for the whole selection or one topic. Maps are saved so they load instantly.
My progress	Mastery per topic, weak topics, quiz history and what to revise next.
Teach me	A short lesson on one topic, written for your level (beginner, intermediate or advanced) and built only from your files.
Delete a source	Removes a file and everything made from it: chunks, topics, quizzes, flashcards and maps.

When a quiz or deck mixes several files, topics are picked evenly from each one, so a huge file can't crowd out a small one.

How it works

Three flows share one database: ingest on upload, ask a question, and generate study tools.



architecture · 3 flows, 12 steps

Uploads are chunked, embedded and stored once. Every question and every study tool starts from the ticked files, and the search step never looks outside them. Video files are transcribed first, so their chunks carry a start time instead of a page.

Tech stack

The backend is a Python FastAPI service on Supabase Postgres with pgvector. Embeddings run on the same machine, and the frontend is plain HTML, CSS and JavaScript with no build step.

Layer	What we used	Why
API	FastAPI, Pydantic, Uvicorn	Routes, request validation and a free `/docs` page for testing
Database	Supabase (hosted PostgreSQL) with psycopg	Raw SQL, no ORM, so the search query stays easy to read
Vector search	pgvector with a SQL function, `match_chunks_multi`	Similarity search limited to the selected files

Layer	What we used	Why
Embeddings	sentence-transformers `all-MiniLM-L6-v2` (384 dimensions)	Runs locally, no API cost
PDF and slides	PyMuPDF, python-pptx	Text plus page and slide numbers
Video	FFmpeg and Whisper (`whisper-large-v3-turbo`)	Audio is split into 10-minute parts and transcribed with timestamps
LLM	One `chat()` wrapper for any OpenAI-compatible API	Swapping the provider only means editing the `.env` file
Frontend	Browser app that calls the API with `fetch`	Sidebar with Course library, Ask your tutor, Practice quiz, Flashcards, Topic map and My progress

How we keep it honest

The search never looks outside your selection. Retrieval calls `match_chunks_multi` with only the ticked ids, so other files aren't just ignored, they're never fetched. Passages scoring below a similarity threshold (0.30 by default, set in `.env`) are thrown away. If nothing is left, the tutor refuses without even calling the LLM.

A second guard in the prompt. The tutor is told to use only the passages it was given, copy the source labels exactly, and reply `NOT_COVERED` when they aren't enough. The code turns that reply into a clean "not covered" message.

One helper checks every selection. `resolve_selection` accepts a list, a single id or a comma-separated string. It removes duplicates, checks each id is a valid UUID, caps the list at 10 and confirms every file exists. Bad input becomes a 400 and an unknown file becomes a 404. Old single-file requests still work.

Same-named topics stay separate. If two files both have a topic called Normalization, the labels become Normalization (Lecture 2) and so on. Mastery and prompts are keyed by topic id, so scores never merge by accident.

Saved results belong to an exact selection. A deck or mind map built from file A is not returned for A plus B. Each different selection gets its own cached map.

Mastery is simple on purpose. Every topic starts at 50. Each correct answer adds 10 and each wrong one takes away 10, within 0 to 100. A score of 40 or below is weak, up to 70 is developing, and above 70 is strong. Auto difficulty maps those bands to easy, medium and hard.

Running it locally

You need Python 3.10 or newer, a Supabase (PostgreSQL) project, an API key for an OpenAI-compatible LLM provider, and FFmpeg if you want to upload videos.

1. Get the code: `git clone https://github.com/anvij229-lang/TutoringAi.git`, then `cd TutoringAi`.
2. Create a `.env` file in `TutoringAI_backend/backend/` with `DATABASE_URL`, `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`, `EMBEDDING_MODEL` and optionally `SIMILARITY_THRESHOLD`. Never commit this file.
3. In the Supabase SQL Editor, run the schema and the multi-file changes: the `match_chunks_multi` function and the `material_ids` columns on `quizzes` and `flashcard_sets`. The PPTX and video changes (`start_seconds`, `source_label`) go in first.
4. Set up and start the backend:
   - `cd TutoringAI_backend/backend`
   - `python3 -m venv .venv` and `source .venv/bin/activate`
   - `python -m pip install -r requirements.txt`
   - `python -m uvicorn app.main:app --reload --port 8001`
5. In a second terminal, from the project root, run `python3 -m http.server 5173` and open `http://localhost:5173/web/`.
6. To try the API directly, open `http://127.0.0.1:8001/docs`, upload two files with different content, and send their ids as `material_ids`.

The first request is slow because the embedding model downloads on first use. Topics are detected once per file and cached, so open the topics endpoint once right after uploading. On a free LLM tier, many new files in a row can cause 429 or 503 errors. Waiting a minute fixes it.

Testing, problems and limits

We tested multi-file mode with two real files: a lecture PDF on adaptive learning systems and a video on database normalization.

Check	Result
Topics for both files	Passed. Six topics from the lecture and four from the video, each with its file title.
Isolation	Passed. With only the lecture selected, the video's question came back not grounded. With both selected, it was answered and the video was cited.
Mixed quiz	Passed. Questions came from both files.
Mixed flashcards	Passed. Sources read like Video 00:00 and Page 1, each with its file name.
Mind map	Partly passed. The two-file map first failed on malformed model replies, so we capped branches at 6 and asked for a smaller tree. A single-file overview (25 nodes) and a single-topic map (18 nodes) both worked.

We also designed the API to return a plain message, never a traceback, for an empty list, a bad id, an unknown id, more than 10 ids, a duplicate id, a topic from an unselected file, a weak-only deck with no weak topics, and a deleted file.

What went wrong along the way

- LLMs sometimes return broken JSON. Mind maps came back truncated or empty, so the code now retries up to three times and validates the result before using it.
- Two routes returned 503 because we passed the list of ids where the function was expected. The `run()` helper takes the function first and the ids second.
- Older test scripts read a `reason` key that the new tutor response no longer has.

Limits today

- A selection is capped at 10 files.
- Quiz and mind map quality depends on the model returning valid JSON, so a request sometimes needs a retry.

- Videos are capped at 30 minutes, and uploads at 30 MB for documents and 200 MB for video.
- Free-tier LLM limits can cause 429 or 503 errors under heavy use.
- There is no login. Users are a plain `user_id` that defaults to `demo`.

What's next

1. Finish wiring every study tool in the frontend to the multi-file endpoints, with one set of checkboxes, a Select all control, and disabled buttons when nothing is ticked.
2. Show the file name next to every citation, using `sources[].citation` and `material_title`.
3. Build an evaluation set that measures how well answers stay grounded, plus integration tests with the frontend.
4. Replace the generic 503 on the mind map and flashcard routes with specific error messages.
5. Add real authentication so mastery and progress belong to actual users.
6. Support more input types and offer a downloadable study pack with the quiz, flashcards and map.